# Data Structures (practicals)

NTIN066 – ÚFAL, 2024/2025

Charles University

---




Adriana Rodríguez Flórez (adrirflorez@gmail.com)

NTIN066 – Data Structures
ÚFAL, 24/25
adrirflorez@gmail.com

# Homework 5:
# ab_experiment

## 1. (a,b)-trees

The random student ID seed used for the execution of the experiments is 179575[13].

### 1.1 Theory

An (a,b)-tree is a multi-way search tree, invented by Bayer and McCreight, with parameters $a \geq 2$ and $b \geq 2a - 1$ [1]. It satisfies the following criteria:

- The root and any internal node has between 2 and b children, except for the leaves, which are external nodes and carry no data.
- Every internal node must have between a and b children, and all external nodes (leaves) are at the same depth.

With regards to basic operations and their complexity, we can establish the following [2][3]:

- When building an (a,b)-tree, it is key to choose the $a$ and $b$ parameters well (especifically, making the latter small), as complexity shall increase with $b$. Now, the minimum dimension constraints we have are $(a, 2a - 1)$: therefore, the smallest dimensions we can work with are (2,3) and (2,4).
- Searching for a key: starting at the root, it goes over the keys stored in the internal nodes [1]. It costs us $\Theta(log\ n \frac{log\ b}{log\ a})$ time, because it involves visiting $O(log_a n)$ nodes and carrying out binary search to compare the target key, in time $O(log\ b.)$
- Insertion and deleting a key: because they both involve the former, regarding amortized costs, they will not perform better than $\Omega(log\ n)$.

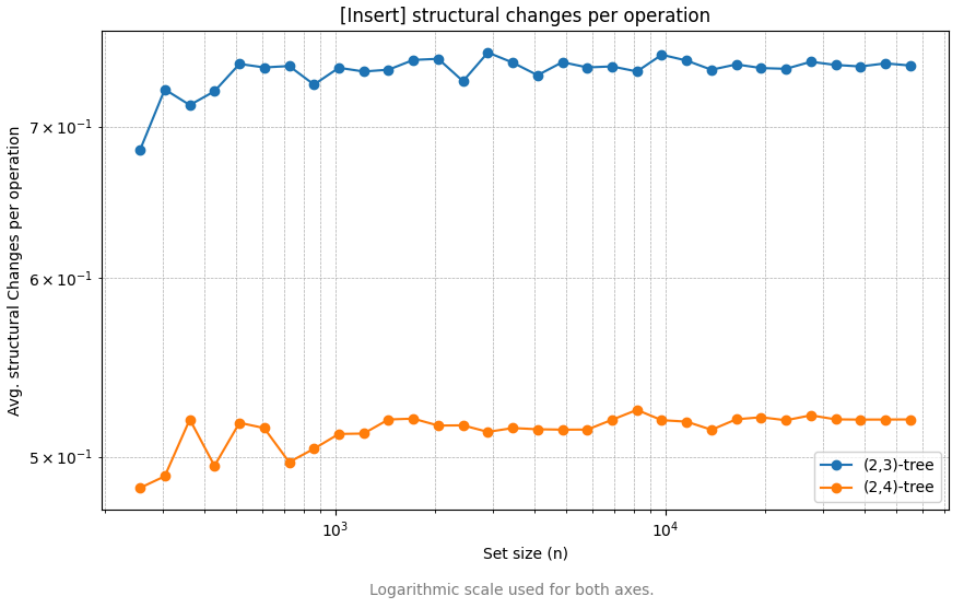
However, the amortized number of modified nodes is constant for some parameters $a$ and $b$ [2][3]:

- a theorem (A) states that a sequence of $m$ insertions on an initially empty (a,b)-tree performs $O(m)$ modifications, as the splits done in the inserts are bounded by the number of nodes.

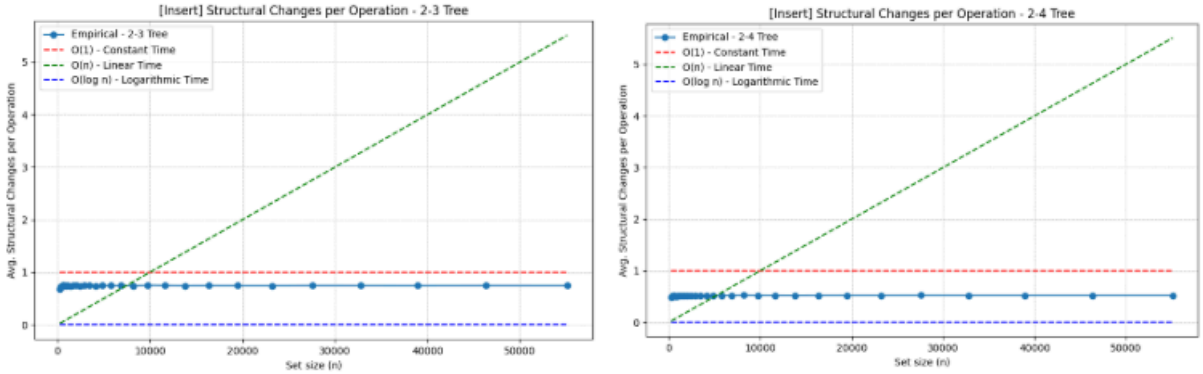
Therefore, it is important to consider the differences in operation costs when choosing parameters:

- In the case an $(a, 2a - 1)$-tree going through through a long sequence of interleaved insert and delete operations, it might suffer oscillations in its structure, each operation with $\Omega(log\ n)$ cost.
- In the case an $(a, 2a)$-tree, having $b \geq 2a$ is enough to maintain the amortized number of changes and avoid this oscillating restructuring of splits and merges. This leads us to another theorem (B), that states that a sequence of $m$ inserts and deletes on an initially empty (a,2a)-tree performs $O(m)$ node modifications.

## 1.2 Insert test




*Plot 1: Insert test results.*




*Plots 2, 3: Insert test results for each tree type, separately.*

The insert test consists of the insertion of n elements in random order. The results for this test show that the average number of structural changes per operation is consistently higher in the (2,3)-tree compared to the (2,4)-tree.

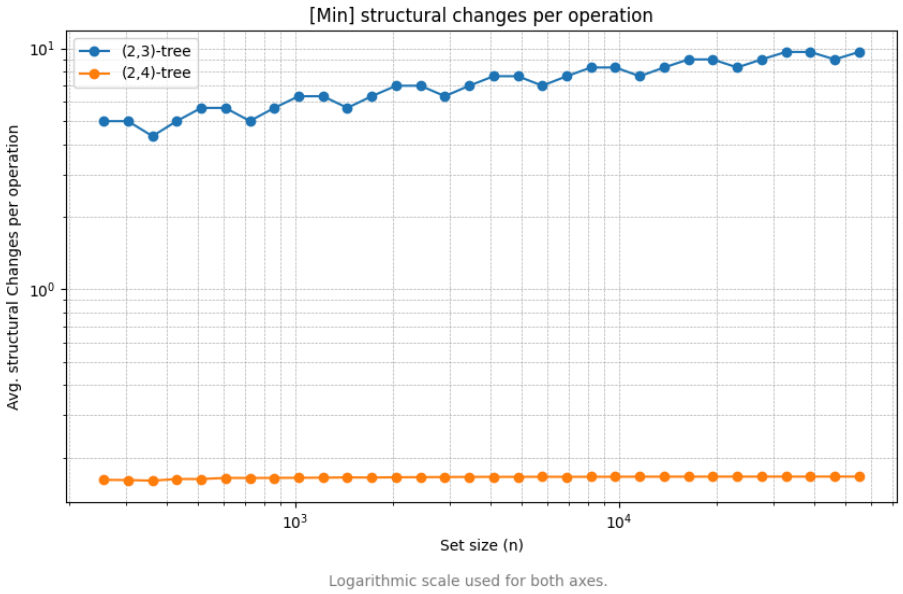
The asymptotic number of modified nodes per operation, for both the (2,3)- and (2,4)-trees, is $O(1)$. They display a constant trend as the set size n grows. This value is ca. $0.72$ modifications per operation on average for the (2,3), with ca. $0.52$ for the (2,4). The resulting complexity aligns with theorem (A), as we expect a constant amortized number of modified nodes for a sequence of m inserts.

About the potential reason why the first shows more modifications than the other, we know that a (2,3)-tree node can hold at most 2 keys (except the leaves, which have no children). On the other hand, a

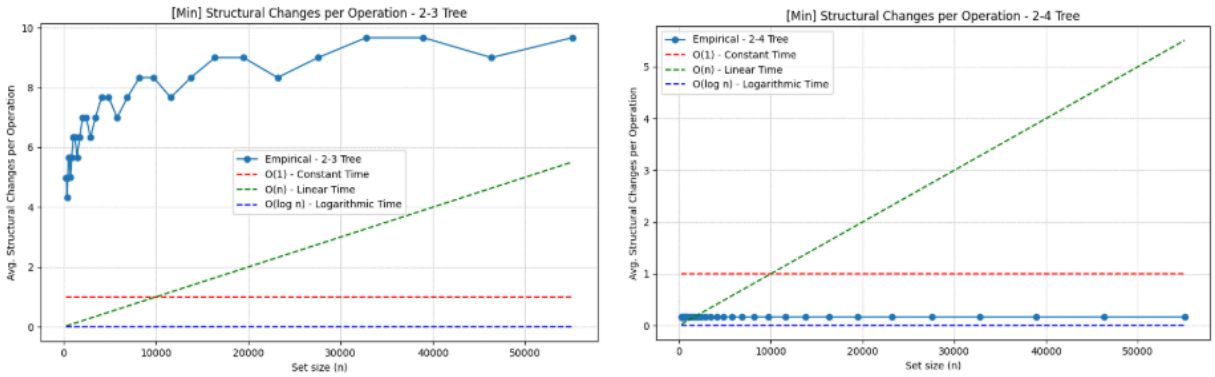
(2,4)-tree can hold at most 3 keys. I believe that the reason why is that the sequential insertions will lead more frequently to node splits when a node reaches maximum key capacity. In contrast, the other tree will experience fewer splits.

Now, when considering the results of this test as a whole, we can see that the tree remains relatively balanced as n grows. Despite being more frequent for (2,3), they are relatively rare, as they only happen when a node becomes oversized (3 children for the first, 4 children for the second).

## 1.3 Min test




*Plot 4: Min test results.*




*Plots 5 and 6: Min test results for each tree type, separately.*

The min test consists in the sequential insertion of n elements, and then repeat the following n times: removing the smallest element in the tree, to then reinsert it.

Firstly, see that for (2,3)-tree, the asymptotic number of modified nodes is $O(log\ n)$ as we can see from the ever-increasing logarithmic curve in plot 5. This clearly aligns with the fact that, for $(a, 2a - 1)$-trees, a sequence of alternating insert and delete operations leads to oscillations in tree structure, yielding $\Omega(log\ n)$ cost per operation.

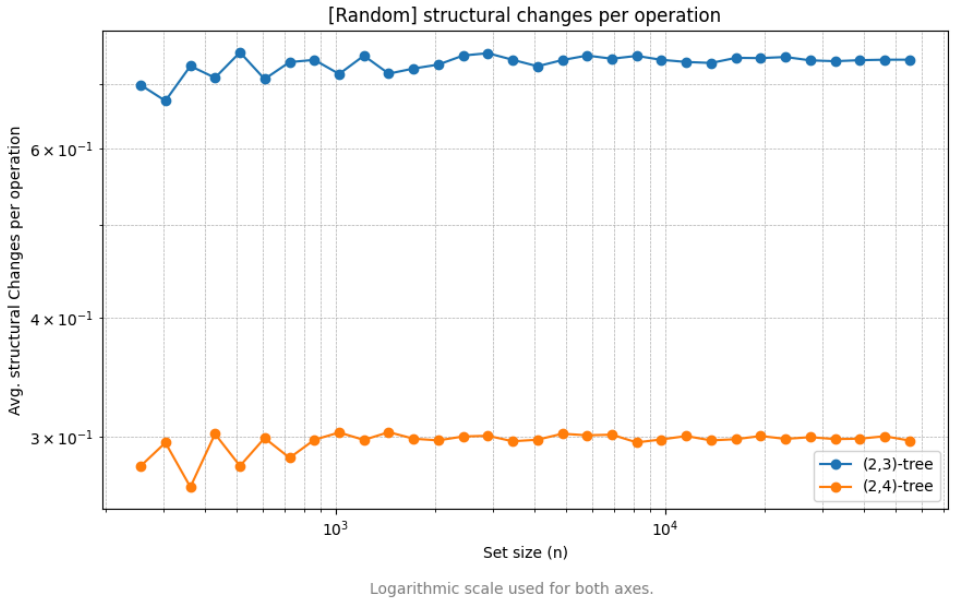
I believe that this has to do with the localization with which the delete operations are taking place: every time we are deleting the smallest (minimal) node, which by definition is the one in the leaf node that is most on the left. Therefore, each time we are emptying an internal node, which provokes a chain of merges that go all the way up.

This leads to the number of node modifications being significantly and increasingly higher as n grows. From the results, this value keeps growing from 1 until convergence to 10 per operation for the (2,3) as n grows, with around $0.25$ on average for the (2,4).

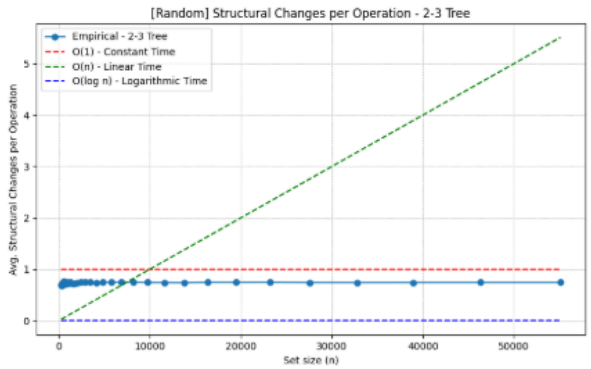
On the other hand, the asymptotic number of modified nodes per operation on the (2,4)-tree is constant $O(1)$. As already established, having a parameter at least $b = 2a$ is a quick solution to the previous issue.

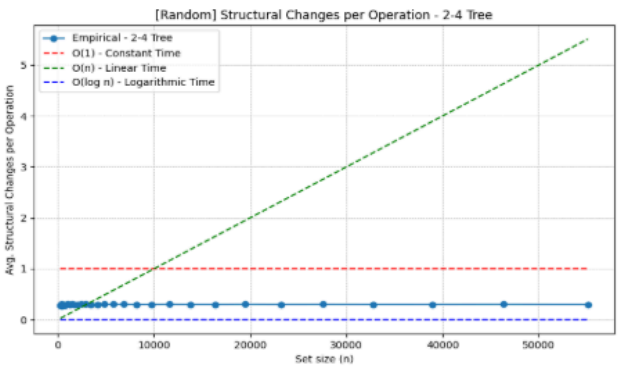
This makes a (2,4)-tree a better candidate design if we want to be able to support long sequences of inserts+deletes without constant merging and splitting. Therefore it aligns with theorem (B), which allows this tree design to keep the amortized number of changes constant.

## 1.4 Random test




*Plot 7: Random test results*




*Plots 8 and 9: Random test results for each tree type, separately.*

The random test consists in the sequential insertion of n elements, and then repeat the following n times: first, remove a random element from the tree, and then reinsert another random element. We assume that the removed element is always present in the tree, and that the inserted element is always not present in the tree.

Like in the first test, both trees show a constant $O(1)$ asymptotic number of modified nodes. On the one hand it corresponds with theory for the (2,4)-tree according to theorem (B), where the b parameter prevents them from going through constant restructuring via splits and mergers when reaching under- or over-capacity.

For the (2,3)-tree the expected trend would be logarithmic due to the interleaving of inserts and deletes and expected $\Omega(log\ n)$ cost per operation, so the experimental results do not correspond with theory.

Nonetheless, the average number of structural changes per operation remains again higher in the (2,3)-tree compared to the (2,4)-tree. The resulting values in this test are around around $0.65$ on average for the (2,3), twice as much than for the (2,4), around $0.3$.

I would say that the main difference with the previous min test results, and the reason why the trend for (2,3) does not correspond with theory, is that the deletes are not localized. What we delete is a random element in the tree, which I believe might make it less likely to provoke a longer sequence of merges across the tree, thus avoiding the expected logarithmic trend which we did see in the min test.

## 2. References

All theoretical facts included in this document have been proven and explained in the study material and lectures of the Data Structures course:


- [1]: Jirka Fink. (2025). Tutorial slides [sl. 31]. Charles University. https://ktiml.mff.cuni.cz/~fink/teaching/data_structures_I/tutorial_01.pdf

- [2]: Martin Mareš. (2024). Lecture notes on Data Structures [pp. 3-8]. Charles University. https://mj.ucw.cz/vyuka/dsnotes/03-abtree.pdf

- [3]: Petr Gregor. (2023). Lecture notes [lectures 3 and 4]. Charles University. https://ktiml.mff.cuni.cz/~gregor/ds1/ds1-l4.pdf https://ktiml.mff.cuni.cz/~gregor/ds1/ds1-l5.pdf